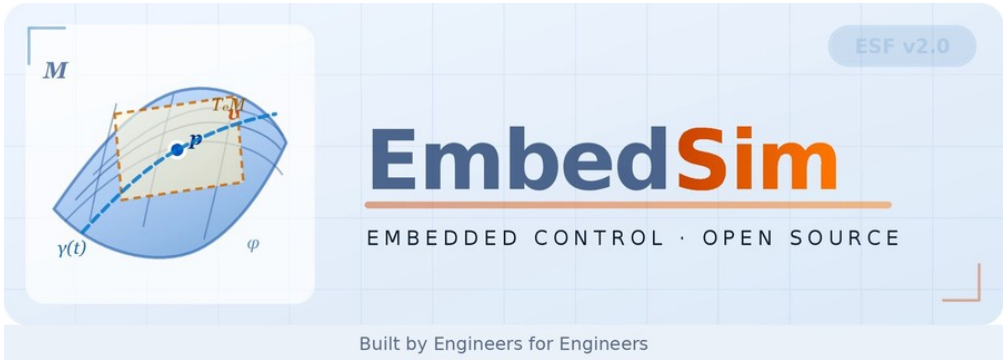




5-Phase Switched Reluctance Motor

SRM Drive Feasibility Study — Based on Platform

Using OpenModelica and EmbedSim

Motor	5-Phase 10/8 SRM — no rare-earth magnets
Vehicle	GVW 673 kg target 45 km/h
Power	4.5 kW peak 48 V DC bus (feasibility target)
Sensor	Resolver → RDC → angle & speed feedback
Plant	OpenModelica model exported as FMU 2.0
Simulation	EmbedSim Python framework (FmuBlock + VectorBlocks)
Controllers	PI (baseline) → Sliding Mode → Differential Flatness
Code output	MISRA C:2012 generated by EmbedSim CodeGen
Process	V-Model development lifecycle

Prerequisites: basic circuit theory, first-year calculus, Python basics
Labs 0 – 6 | OpenModelica · EmbedSim · AURIX C code

How to Use This Guide

This guide walks you through building and simulating a complete motor drive system for a Sri Lankan three-wheeler (tuk-tuk). You will use two tools:

- OpenModelica — a free, open-source tool to model the motor and inverter as a physical system.
- EmbedSim — a Python simulation framework to implement the controller and run the closed-loop simulation.

Each section contains a numbered TASK. Every task has:

Symbol	Meaning
 HINT	A nudge in the right direction — read this before you get stuck, not after.
 WATCH OUT	A common mistake made by first-time students — avoid this.
 CHECK	A quick sanity check you can do to confirm you are on the right track.
 TIP	A practical shortcut or good engineering habit.
 EXPECTED RESULT	What your simulation output should look like when done correctly.
 PHYSICS LINK	Connects what you just coded to a real physical phenomenon — always read these.

Do not copy-paste code blindly. Read each line, understand what it does, then type it. You will learn nothing from pasting.

Lab 0 — Feasibility Study: Does an SRM Work for a Tuk-Tuk?

This is a feasibility study — the first step of the V-Model. Before any design begins, an engineer must answer: is the concept physically possible? Can an SRM of reasonable size deliver what a tuk-tuk actually needs? You will derive the requirements from real vehicle data, calculate whether a 5-phase SRM can meet them, and decide whether to proceed. No simulation yet — just mechanics, a calculator, and your engineering judgement.

Determine the minimum motor power and torque required for an SRM tuk-tuk drive on Sri Lankan roads, using the as the reference vehicle. Output: a requirements table that becomes the acceptance criteria for all simulation work in Labs 1–5.

Every motor choice starts with a force balance on the vehicle. Torque and power are not chosen arbitrarily — they are derived from Newton's laws applied to the specific vehicle, road, and duty cycle. If your simulation speed or torque does not match these numbers, your model is wrong.

0.1 Reference Vehicle — Platform Data

These numbers come directly from published specifications. The SRM drive must match or exceed the performance of the existing ICE drivetrain where practical, and at minimum match the performance of the version.

In V-Model terms this is the System Requirements level — the top of the left side. Every parameter below is traceable to a physical constraint of the vehicle. The feasibility study begins here.

Parameter	Symbol	Value	Unit
kerb weight (vehicle only)	m_kerb	337	kg [
Driver + 3 passengers (typical)	m_pass	300	kg (75 kg × 4 occupants)
Gross vehicle mass (GVW = kerb + payload used)	m	637	kg (kerb + occupants ;
Wheel radius — tyre 4.00-8, loaded	r_w	0.27	m (8-inch rim + tyre section)
Rolling resistance coefficient (tarmac)	C_rr	0.015	— (slightly high: Sri Lankan road surface)
Aerodynamic drag area ($C_d \times$ frontal area)	$C_d A$	1.4	m ² (measured frontal area ~1.6

Parameter	Symbol	Value	Unit
			m ² , C _d ≈ 0.9)
Air density (Colombo, sea level, 32°C)	ρ	1.17	kg/m ³
Gravitational acceleration	g	9.81	m/s ²
Drivetrain efficiency (chain + bearing losses)	η	0.88	—
Chain reduction ratio	G	4	— (to be confirmed in design)
Target top speed (matches)	v _{max}	45	km/h
Hill grade — Colombo to Kandy approach roads	i	15	% (conservative for feasibility)
Acceleration: 0 to 40 km/h in	t _{acc}	10	s (relaxed — torque is instant)
DC bus voltage (touch-safe LiFePO ₄ pack)	V _{dc}	48	V
reference power	P _{ref}	4.5	kW (target)
reference torque	T _{ref_whl}	36	N·m at wheel (reference gradeability 29%)

0.2 The Three Forces Acting on the Tuk-Tuk

Three forces resist motion. You must provide enough motor torque to overcome all three simultaneously in the worst case.

Force 1 — Rolling Resistance

Every tyre deforms as it rolls. The energy lost to this deformation appears as a drag force opposing motion:

$$F_{\text{roll}} = C_{\text{rr}} \times m \times g$$

Calculate F_{roll} using the values in the table above. Write your answer here:

$$F_{\text{roll}} = \underline{\hspace{2cm}} \text{ N}$$

Rolling resistance is nearly constant with speed — it costs the same energy per metre whether you are going 5 km/h or 50 km/h. It is the dominant force at low speed.

Force 2 — Aerodynamic Drag

Air resistance grows with the square of speed. The drag force is:

$$F_{\text{aero}} = 0.5 \times \rho \times C_d A \times v^2$$

Calculate F_{aero} at the two speeds below. v must be in m/s (divide km/h by 3.6).

Speed	v (m/s)	F_{aero} (N) — you calculate
20 km/h	$= 20 / 3.6 = \underline{\hspace{2cm}}$ m/s	$F_{\text{aero}} = \underline{\hspace{2cm}}$ N
45 km/h	$= 45 / 3.6 = \underline{\hspace{2cm}}$ m/s	$F_{\text{aero}} = \underline{\hspace{2cm}}$ N

Aerodynamic drag grows as v^2 . Doubling the speed quadruples the drag. At 45 km/h it is already significant on a box-shaped tuk-tuk ($C_d \approx 0.9$ vs 0.3 for a car). This is why the tops out at 45 km/h — pushing beyond that would need roughly 2× the motor power for only 10 km/h gain.

Force 3 — Hill Climbing

On a slope of grade i (rise/run as a fraction), the gravitational component along the road is:

$$F_{\text{grade}} = m \times g \times \sin(\arctan(i))$$

For $i = 10\% = 0.10$, $\arctan(0.10) \approx 5.71^\circ$. Calculate F_{grade} :

$$F_{\text{grade}} = \underline{\hspace{2cm}} \text{ N}$$

On a gentle grade, $\sin(\arctan(i)) \approx i$, so $F_{\text{grade}} \approx m \times g \times i$. This approximation is accurate to within 0.5% for grades below 15%.

Force 4 — Acceleration

To accelerate the vehicle (treat the tuk-tuk as a point mass for now):

$$a = \Delta v / t_{\text{acc}} = (40/3.6) / 8 = \underline{\hspace{2cm}} \text{ m/s}^2$$

$$F_{\text{accel}} = m \times a = \underline{\hspace{2cm}} \text{ N}$$

This is Newton's second law: $F = ma$. The motor must supply this force on top of all the road-load forces while accelerating. It defines the peak torque requirement.

TAS
K 0.1

Calculate required wheel force and shaft torque

Complete this table. The worst-case condition for each row is shown — use the forces you calculated above.

Operating condition	Forces to add	Total F at wheel (N)	Torque at wheel T_w (N·m)	Motor torque $T_m = T_w / (G \times \eta)$ (N·m)
Cruise at 45 km/h (flat)	$F_{\text{roll}} + F_{\text{aero}}(45)$	$\underline{\hspace{2cm}}$	$T_w = F \times r_w = \underline{\hspace{2cm}}$	$T_m = \underline{\hspace{2cm}}$
Hill climb at 20 km/h (10%)	$F_{\text{roll}} + F_{\text{grade}}$	$\underline{\hspace{2cm}}$	$T_w = \underline{\hspace{2cm}}$	$T_m = \underline{\hspace{2cm}}$
Acceleration 0–40 km/h (flat)	$F_{\text{roll}} + F_{\text{accel}}$	$\underline{\hspace{2cm}}$	$T_w = \underline{\hspace{2cm}}$	$T_m = \underline{\hspace{2cm}}$
Worst case: accel on hill	$F_{\text{roll}} + F_{\text{grade}} + F_{\text{accel}}$	$\underline{\hspace{2cm}}$	$T_w = \underline{\hspace{2cm}}$	$T_m = \underline{\hspace{2cm}}$

$T_w = F_{\text{total}} \times r_w$ (wheel radius in metres). $T_m = T_w \div (G \times \eta)$ where $G = 4$ is the gear ratio — the motor spins 4 times faster than the wheel so it only needs 1/4 of the wheel torque (times efficiency correction).

TAS
K 0.2

Calculate required power

Power = Force × velocity. The motor must supply this at the shaft:

$$P_{\text{shaft}} = F_{\text{total}} \times v / \eta$$

Complete the table:

Operating condition	v (m/s)	F _{total} (N)	P _{shaft} = F × v / η (W)
Cruise at 45 km/h	= _____	= _____	P = _____
Hill climb at 20 km/h	= _____	= _____	P = _____
Acceleration (peak, at 40 km/h)	= _____	= _____	P = _____
	11.1 m/s	36 N·m at wheel	4.5 kW — is your answer close?

Power and torque are different things. A motor can have high torque at low speed (useful for hills and acceleration) but low power. Or high power at high speed but low torque. The tuk-tuk needs both: high torque for starting, enough power for cruise.

TAS
K
0.2b

Feasibility verdict

Compare your calculated power to the (4.5 kW). Fill in the verdict:

Condition	Your P _{shaft} (W)	reference	Feasible with 4.5 kW SRM?
Cruise at 45 km/h	_____	≈ 2 800 W (cruise estimate, 45 km/h flat)	YES / NO / MARGINAL
Hill climb at 15% grade	_____	29% gradeability typical for this class	YES / NO / MARGINAL
Peak acceleration	_____	36 N·m wheel torque — reference value for this class	YES / NO / MARGINAL

If your cruise power is under 4.5 kW and your hill-climb power is under 4.5 kW individually, the feasibility study passes. The motor does not have to do both simultaneously at full load — but it must handle each worst case.

Write one sentence here: Based on the and your calculations, a 5-phase SRM rated at _____ kW / _____ N·m at the motor shaft is [FEASIBLE / NOT FEASIBLE] for this application. This sentence is your V-Model system-level requirement statement.

TAS
K 0.3

Verify motor physics in simulation

Once you have your numbers, use them as checkpoints throughout Labs 1–5:

What to check	Where to check it	Pass condition
Peak motor torque T_m	Lab 3 open-loop, torque signal	Simulation peak $\approx$ your T_m (worst case row)
Shaft power at cruise	Lab 4 PI controller, speed $\times T_{ref}$	$P_{sim} \approx$ your P_{shaft} (cruise row)
Motor speed at 45 km/h	Lab 4, steady-state RPM	$rpm = v_{max} \times G / (2\pi \times r_w) \times 60 = ?$ (compare to Quick Ref)
Acceleration time	Lab 5, time to reach 40 km/h	$t_{acc} \leq 8$ s
Hill load (15% grade)	Lab 5 Scenario B, $T_{load} = F_{grade} \times r_w / G$	Speed dip < 200 RPM — use your F_{grade} value

This is what motor control engineers do on every project: derive the load requirements from mechanics, then verify the simulation matches. If your simulation shows 5 N·m torque but your calculation says 20 N·m is needed, either the gains are wrong or the plant parameters are wrong. Physics never lies.

0.4 Motor Speed Calculation

The motor shaft speed at vehicle top speed depends on the gear ratio and wheel size. Derive the formula yourself, then fill in the blank:

$$n_{motor} = v_{max} \times G / (2\pi \times r_w) \times 60 \text{ [RPM]}$$

$$n_{motor} = \underline{\hspace{2cm}} \text{ RPM at } v_{max} = 45 \text{ km/h}$$

Now cross-check: the motor produces 36 N·m of torque. Through your gear ratio G and wheel radius r_w , what wheel torque does that correspond to? Does your required wheel torque (from Task 0.1) fit within that budget?

$$T_{wheel} = T_{motor} \times G \times \eta = \underline{\hspace{2cm}} \text{ N}\cdot\text{m (your SRM at rated torque)}$$

Compare to : 36 N·m — is your design in the same ballpark?

v_{max} must be in m/s. Divide km/h by 3.6 first.

Background — What Is a Switched Reluctance Motor?

Before you start simulating, you need to understand what you are simulating. Spend 15 minutes reading this section carefully.

The Key Idea

A Switched Reluctance Motor (SRM) has no magnets and no windings on the rotor. The rotor is just shaped steel. Torque is produced because magnetic flux always tries to take the path of least resistance (least reluctance). When a stator coil is energised, the nearest rotor tooth is pulled towards alignment — this is the force that spins the motor.

Think of a magnet attracting a steel nail. The nail is pulled towards the magnet — that pull is reluctance torque. The SRM does this continuously, phase by phase, to keep spinning.

Why 5 Phases and 10/8 Poles?

Our motor has 10 stator poles and 8 rotor poles, arranged in 5 pairs (phases). With 5 phases, there is always at least one phase producing torque no matter where the rotor sits. This avoids dead spots and gives smooth rotation.

Phase	Stator poles	Rotor poles attracted	Electrical period
1	1 and 6	1 and 5	Every 45°
2	2 and 7	2 and 6	Every 45°
3	3 and 8	3 and 7	Every 45°
4	4 and 9	4 and 8	Every 45°
5	5 and 10	5 and 1	Every 45°

The Phase Voltage Equation

When you apply a voltage v to one phase, the current i obeys:

$$v = R \cdot i + L(\theta) \cdot (di/dt) + i \cdot (dL/d\theta) \cdot \omega$$

where R is winding resistance, $L(\theta)$ is inductance (which changes with rotor angle θ), and ω is rotor speed. The last term is the back-EMF — it opposes the current when the motor is spinning.

The torque produced by one phase is:

$$T = 0.5 \cdot i^2 \cdot dL/d\theta$$

This is positive (motoring) when inductance is rising, and negative (braking/regenerating) when inductance is falling. The controller must switch phases on and off at the right angles to keep torque positive.

Notice that torque depends on i^2 — so doubling the current gives four times the torque. But also four times the copper loss (heat). This is why current limits matter.

Lab 1 — Build the OpenModelica Phase Model

You will build the model of one SRM phase in OpenModelica and check that it behaves correctly when you apply a voltage step.

TASK 1.1

Create a new Modelica model file

1. Open OpenModelica (OMEdit). Go to File → New Model.
2. Name it SRM_Phase. Set it as a model (not a package).
3. Add the following parameters at the top of the model:

```
parameter Real R = 0.045 "Phase resistance [Ohm]";
parameter Real Lu = 0.9e-3 "Unaligned inductance [H]";
parameter Real La = 4.5e-3 "Aligned inductance [H]";
parameter Real Nr = 8 "Number of rotor poles";
```

Parameters in Modelica are constants you set before the simulation runs. Use descriptive names with units in the string comment — it saves confusion later.

Make sure you write 0.9e-3, not 0.9. Modelica expects SI units. 0.9 Henrys would be a huge motor — you would get very strange results.

TASK 1.2

Add the inductance function

The inductance L varies with rotor angle. We use a simple piecewise-linear (triangular) profile. Add this function inside your model:

```
function inductanceProfile
  input Real theta "Rotor angle [rad]";
  input Real Lu "Unaligned inductance [H]";
  input Real La "Aligned inductance [H]";
  input Real Nr "Number of rotor poles";
  output Real L "Inductance [H]";
protected
  Real pitch, theta_norm, f;
algorithm
  pitch := 2*Modelica.Constants.pi / Nr;
  theta_norm := mod(theta, pitch) / pitch; // 0..1 within one pole pitch
  if theta_norm < 0.4 then
    f := theta_norm / 0.4; // rising: 0 to 1
  elseif theta_norm < 0.6 then
    f := 1.0; // aligned (peak)
  else
    f := (1.0 - theta_norm) / 0.4; // falling: 1 to 0
  end if;
  L := Lu + (La - Lu) * f;
```

```
end inductanceProfile;
```

The function maps any rotor angle to an inductance value. The 0.4 / 0.6 split means: 40% of the pole pitch is rising, 20% is at peak, 40% is falling. You can tune these fractions to match a real motor measured on a test bench.

Plot L vs θ from 0 to 2π . You should see 8 identical triangular peaks (one per rotor pole), oscillating between 0.9 mH and 4.5 mH.

TAS K 1.3

Write the phase voltage equation

Now add the dynamic equations. In the equation section of SRM_Phase, write:

```
// State variable: phase current
Real i_ph(start=0.0) "Phase current [A]";
Real L_ph "Phase inductance [H]";
Real dLdTheta "dL/dθ [H/rad]";
Real T_ph "Phase torque [N·m]";
input Real v_ph "Phase voltage [V]";
input Real theta "Rotor angle [rad]";
input Real omega "Rotor speed [rad/s]";
equation
  L_ph = inductanceProfile(theta, Lu, La, Nr);
  dLdTheta = (inductanceProfile(theta+1e-4, Lu, La, Nr)
    - inductanceProfile(theta-1e-4, Lu, La, Nr)) / 2e-4;
  v_ph = R * i_ph + L_ph * der(i_ph) + i_ph * dLdTheta * omega;
  T_ph = 0.5 * i_ph^2 * dLdTheta;
```

We estimate $dL/d\theta$ numerically using a tiny step of $1e-4$ radians (≈ 0.006 degrees). This is called a finite-difference derivative. It is accurate enough for simulation purposes.

The equation $v = R \cdot i + L \cdot di/dt + i \cdot (dL/d\theta) \cdot \omega$ is an ODE (ordinary differential equation). The `der()` function in Modelica means d/dt . OpenModelica will solve this automatically — you do not need to discretise it yourself.

With $v_{ph} = 48$ V, $\theta = 0.3$ rad (rising inductance region), $\omega = 0$ (motor stopped), simulate for 5 ms. The current should rise from 0 A and approach $48/0.045 \approx 1067$ A asymptotically — but clip it in the next step!

Lab 2 — Assemble the Full Motor and Export as FMU

You will connect 5 phase models to a rotor mechanics block and export the complete plant as an FMU for use in EmbedSim.

TAS K 2.1

Build the rotor mechanics block

Create a new model called SRM_Rotor with these equations:

```
parameter Real J = 0.010 "Combined rotor+wheel inertia [kg·m²]";
parameter Real B = 0.002 "Viscous friction [N·m·s/rad]";
Real omega(start=0) "Rotor speed [rad/s]";
Real theta(start=0) "Rotor angle [rad]";
input Real T_net "Net torque [N·m]";
equation
  J * der(omega) = T_net - B * omega;
  der(theta) = omega;
```

This is Newton's second law for rotation: torque = inertia × angular acceleration. The $J \cdot d\omega/dt = T_{\text{net}}$ equation will accelerate the rotor when net torque is positive, and decelerate it when negative.

TAS K 2.2

Assemble the top-level model

Create SRM_Plant_10_8. Instantiate 5 copies of SRM_Phase and 1 copy of SRM_Rotor:

```
model SRM_Plant_10_8
  SRM_Phase phase1, phase2, phase3, phase4, phase5;
  SRM_Rotor rotor;
  input Real v_in[5] "Phase voltages from inverter [V]";
  input Real T_load "Load torque from vehicle [N·m]";
  output Real omega_out "Rotor speed [rad/s]";
  output Real theta_out "Rotor angle [rad]";
  output Real i_out[5] "Phase currents [A]";
equation
  // Connect voltages (each phase offset by 1/5 of the pole pitch)
  phase1.v_ph = v_in[1]; phase1.theta = rotor.theta;
  phase2.v_ph = v_in[2]; phase2.theta = rotor.theta + 2*pi/5;
  phase3.v_ph = v_in[3]; phase3.theta = rotor.theta + 4*pi/5;
  phase4.v_ph = v_in[4]; phase4.theta = rotor.theta + 6*pi/5;
  phase5.v_ph = v_in[5]; phase5.theta = rotor.theta + 8*pi/5;
  for k in 1:5 loop phaseN.omega = rotor.omega; end for;
  // Sum torques and drive rotor
  rotor.T_net = phase1.T_ph+phase2.T_ph+phase3.T_ph
  +phase4.T_ph+phase5.T_ph - T_load;
  // Outputs
  omega_out = rotor.omega;
  theta_out = rotor.theta;
  for k in 1:5 loop i_out[k] = phaseN.i_ph; end for;
```

```
end SRM_Plant_10_8;
```

The phase angle offsets (0 , $2\pi/5$, $4\pi/5$, ...) ensure that the 5 phases are evenly spaced around the cycle. Without this, all 5 phases would fire simultaneously and you would get huge torque pulses rather than smooth rotation.

The loop syntax `phaseN.omega` refers to each phase instance. In Modelica you cannot use array index notation directly on component instances — you must either connect each one individually, or use an array of components: `SRM_Phase phases[5]`.

**TAS
K 2.3**

Export as FMU 2.0

4. In OMEdit, open `SRM_Plant_10_8`.
5. Go to `FMI` → `Export FMU`.
6. Choose: `FMI Version 2.0`, `Type = Co-Simulation`.
7. Click `Export`. OpenModelica will compile the model to a `.fmu` file.
8. Copy the `.fmu` file to your EmbedSim project folder.

The FMU is a self-contained binary — it includes the compiled model equations and a description XML file. EmbedSim's `FmuBlock` reads this file automatically. You do not need to have OpenModelica installed on the simulation computer.

Unzip the `.fmu` file (it is just a ZIP archive) and open `modelDescription.xml`. Verify that you can see the variable names `v_in`, `T_load`, `omega_out`, `theta_out`, `i_out` listed as `ScalarVariable` entries.

Lab 3 — Open-Loop Test in EmbedSim

Before adding a controller, you will run the plant open-loop with a fixed voltage pattern to check that it spins correctly.

TAS K 3.1

Set up your EmbedSim project

Create a new folder called `srm_tukTuk/` and copy your `.fmu` file into it. Then create `srm_openloop_test.py`:

```
# srm_openloop_test.py
import numpy as np
from embedsim.simulation_engine import SimulationEngine
from embedsim.block_topology import BlockTopology
from embedsim.fmu_blocks import FmuBlock
from embedsim.simulation_logger import SimulationLogger

DT = 50e-6 # 50 microsecond time step
T_END = 2.0 # simulate 2 seconds

plant = FmuBlock('SRM_Plant_10_8.fmu', dt=DT)
topo = BlockTopology()
topo.add_block(plant)
logger = SimulationLogger(topo)
engine = SimulationEngine(topo, logger, dt=DT, t_end=T_END)
engine.run()
logger.plot(['omega_out', 'i_out_0'])
```

`FmuBlock` wraps the OpenModelica FMU so it looks like any other EmbedSim block. The `dt` argument must match the step time you use in `SimulationEngine` — otherwise the FMU internal integrator will be out of sync with EmbedSim.

If you get 'FMU file not found', check that the `.fmu` file is in the same folder as your Python script, or provide the full file path.

TAS K 3.2

Apply a fixed commutation pattern

In open-loop, we drive the phases with a simple fixed-angle switching table. Create `srm_openloop_commutation.py`:

```
import numpy as np

def openloop_voltages(theta, V_dc=48.0, Nr=8, q=5):
    """Return v_phase[5] for open-loop fixed-angle commutation."""
    pitch = 2 * np.pi / Nr # 45 degrees in radians
    v_out = np.zeros(5)
    for k in range(q):
        theta_k = theta - k * (2*np.pi / q) # phase offset
```

```

theta_k = theta_k % pitch # within one pitch
if 0.0 <= theta_k <= 0.26: # excitation window
    v_out[k] = V_dc
elif 0.26 < theta_k <= 0.38: # freewheeling
    v_out[k] = 0.0
else:
    v_out[k] = -V_dc # demagnetise
return v_out

```

In your main simulation loop, call this function each step:

```

# Inside engine step callback:
theta = plant.get_output('theta_out')
v_phase = openloop_voltages(theta)
plant.set_input('v_in', v_phase)
plant.set_input('T_load', 5.0) # 5 N·m constant load

```

The numbers 0.26 and 0.38 radians define the turn-on and turn-off angles within one rotor pole pitch of 0.785 rad (45°). These are the most important tuning parameters of an SRM drive — changing them changes efficiency and torque ripple significantly.

With $V_{dc} = 48$ V and $T_{load} = 5$ N·m, the motor should accelerate from 0 to roughly 150–200 rad/s (≈ 1400 –1900 RPM) within 1 second, then settle. The current should show 5 overlapping pulses per rotor pole pitch, each reaching 30–50 A.

TAS K 3.3

Plot and analyse the open-loop results

9. Plot rotor speed (ω_{out}) vs time. Does it reach steady state?
10. Plot all 5 phase currents on the same graph. Are they evenly spaced in time?
11. Compute the torque ripple: $(T_{max} - T_{min}) / T_{mean} \times 100\%$. Record this value — you will compare it to the closed-loop result in Lab 5.

Use `logger.plot(['i_out_0','i_out_1','i_out_2','i_out_3','i_out_4'])` to see all 5 currents. In a healthy 5-phase SRM, they should look like identical pulses shifted by $1/5$ of the period.

Lab 4 — Add a PI Speed Controller

You will implement a simple PI (Proportional-Integral) speed controller and observe how the tuk-tuk motor responds to a speed reference.

What Does a PI Controller Do?

A PI controller produces a control output u from an error signal $e = \text{reference} - \text{measured}$:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt$$

The proportional term $K_p \cdot e$ gives an immediate response. The integral term $K_i \cdot \int e dt$ removes the steady-state error — if the motor is spinning slightly too slowly, the integral builds up and increases the torque command until the error is zero.

Think of K_p as how hard you press the accelerator when you see you are going too slowly. Think of K_i as how long you keep pressing — the longer the error persists, the more you push.

TAS
K 4.1

Implement the PI speed controller block

Create `srm_pi_controller.py`:

```
from embedsim.vector_block import VectorBlock

class SRM_PI_SpeedController(VectorBlock):
    """PI speed controller for SRM tuk-tuk."""

    def __init__(self, Kp=0.05, Ki=0.5, T_max=25.0):
        super().__init__()
        self.Kp = Kp # [N.m.s/rad]
        self.Ki = Ki # [N.m/rad]
        self.T_max = T_max # torque clamp [N.m]
        self._integ = 0.0 # integrator state

    INPUT_NAMES = ['omega_ref', 'omega_meas']
    OUTPUT_NAMES = ['T_ref']

    def step(self, dt):
        e = self.omega_ref - self.omega_meas
        self._integ += e * dt
        T_raw = self.Kp * e + self.Ki * self._integ
        # Clamp torque: only positive (motoring), no braking yet
        self.T_ref = max(0.0, min(T_raw, self.T_max))
        # Anti-windup: freeze integrator at limits
        if T_raw != self.T_ref:
            self._integ -= e * dt # undo last integration
```

The anti-windup logic (last 2 lines) prevents the integrator from accumulating error when the output is already at its limit. Without it, the motor will overshoot badly when the speed reference is first applied.

The torque clamp `max(0.0, ...)` means the motor can only push, not brake. In Lab 5 you will remove this restriction to enable regenerative braking.

TAS K 4.2

Connect the controller in EmbedSim

Create `srm_pi_closedloop.py`:

```
from embedsim.simulation_engine import SimulationEngine
from embedsim.block_topology import BlockTopology
from embedsim.fmu_blocks import FmuBlock
from embedsim.simulation_logger import SimulationLogger
from srm_pi_controller import SRM_PI_SpeedController
from srm_commutation_block import SRM_CommutationBlock

DT = 50e-6; T_END = 5.0
OMEGA_REF = 2400 * 2*3.14159 / 60 # 2400 RPM in rad/s

plant = FmuBlock('SRM_Plant_10_8.fmu', dt=DT)
pi = SRM_PI_SpeedController(Kp=0.05, Ki=0.5)
comm = SRM_CommutationBlock(Nr=8, q=5)

topo = BlockTopology()
topo.add_blocks([plant, pi, comm])

# Wire the signals
topo.connect(plant, 'omega_out', pi, 'omega_meas')
topo.connect(pi, 'T_ref', comm, 'T_ref')
topo.connect(plant, 'theta_out', comm, 'theta_meas')
topo.connect(comm, 'v_phase', plant, 'v_in')

pi.omega_ref = OMEGA_REF # constant speed reference
plant.set_input('T_load', 8.0) # tuk-tuk road load at 30 km/h

logger = SimulationLogger(topo)
engine = SimulationEngine(topo, logger, dt=DT, t_end=T_END)
engine.run()
logger.plot(['omega_ref_pi', 'omega_out', 'T_ref'])
```

The `topo.connect()` calls define the data-flow wiring. Think of it like connecting wires on a circuit: the output port of one block feeds into the input port of the next. EmbedSim enforces that no port is connected twice.

TAS K 4.3

Tune the PI gains

Run the simulation three times with different `Kp` values and record the results:

Kp	Ki	Overshoot (%)	Settling time (s)	Steady-state error (RPM)
0.01	0.5	?	?	?
0.05	0.5	?	?	?
0.20	0.5	?	?	?

Then tune Ki with your best Kp:

Kp	Ki	Overshoot (%)	Settling time (s)	Steady-state error (RPM)
0.05	0.1	?	?	?
0.05	0.5	?	?	?
0.05	2.0	?	?	?

A good tuning goal: settling time < 1.5 s, overshoot < 10%, zero steady-state error. If you see oscillations that do not die away, Kp is too large. If the motor reaches speed but stays 50–100 RPM below target, Ki is too small.

With Kp = 0.05 and Ki = 0.5: expect settling $\approx$ 1.2 s, overshoot $\approx$ 8%, steady-state error < 5 RPM.

Lab 5 — Sliding Mode Controller and Comparison

You will replace the PI controller with a Sliding Mode Controller (SMC), run identical test scenarios, and compare performance.

What Is a Sliding Mode Controller?

An SMC defines a sliding surface s — a function of the error that equals zero when the system is behaving correctly. The controller switches aggressively to push the system onto this surface, then keeps it there.

$$s = e_{\omega} + c \cdot \int e_{\omega} dt$$

where $e_{\omega} = \omega_{ref} - \omega_{meas}$

Once on the surface ($s = 0$), the system slides along it to the desired operating point. The SMC is much more robust to load disturbances than a PI — useful when the tuk-tuk hits a pothole or a hill.

Imagine you are steering a car. PI is like gently turning the wheel when you drift off course. SMC is like firmly grabbing the wheel and snapping it back the moment you deviate — more aggressive, more precise.

TAS
K 5.1

Implement the SMC block

Create `srm_smc_controller.py`:

```
import numpy as np
from embedsim.vector_block import VectorBlock

class SRM_SMC_SpeedController(VectorBlock):
    def __init__(self, J=0.010, B=0.002, c=20.0, Ksw=5.0, phi=0.05):
        super().__init__()
        self.J = J # rotor inertia [kg.m²]
        self.B = B # friction [N.m.s/rad]
        self.c = c # surface slope [rad/s]
        self.Ksw = Ksw # switching gain [N.m]
        self.phi = phi # boundary layer [rad/s]
        self._integ = 0.0
        self._T_load_est = 0.0 # disturbance estimate

    INPUT_NAMES = ['omega_ref', 'omega_meas', 'omega_ref_dot']
    OUTPUT_NAMES = ['T_ref']

    def step(self, dt):
        e = self.omega_ref - self.omega_meas
        self._integ += e * dt
        s = e + self.c * self._integ
        # Equivalent (feed-forward) control
        T_eq = (self.J * self.omega_ref_dot
                + self.B * self.omega_meas
                + self._T_load_est)
        # Switching control with saturation (reduces chattering)
```

```

if abs(s) < self.phi:
    sat_s = s / self.phi
else:
    sat_s = np.sign(s)
T_sw = -self.Ksw * sat_s
T_raw = T_eq + T_sw
self.T_ref = np.clip(T_raw, -25.0, 25.0)
# Simple first-order load observer
self._T_load_est += 50.0 * (T_raw - self.T_ref) * dt

```

The `sat()` function (the if/else block) replaces the hard `sign()` function with a smooth ramp inside the boundary layer `phi`. This reduces chattering — the high-frequency switching noise that `sign()` produces. Real motors cannot respond to infinite-bandwidth switching, so smooth it out.

The `omega_ref_dot` input (rate of change of speed reference) requires your reference generator to compute a derivative. For a simple step reference, `omega_ref_dot = 0` except at the moment of the step. For a smooth ramp reference, `omega_ref_dot = ramp slope`.

TAS K 5.2

Run the three test scenarios

Run both PI and SMC controllers on these three scenarios. Fill in the results table.

Scenario A — Speed step:

- `omega_ref`: 0 → 2400 RPM step at $t = 0$.
- `T_load`: 8 N·m constant.
- Measure: rise time (0 to 90% of final speed), overshoot, settling time.

Scenario B — Load disturbance:

- `omega_ref`: constant 2400 RPM.
- `T_load`: 8 N·m, then step to 18 N·m at $t = 2$ s (tuk-tuk hits a hill).
- Measure: speed dip (RPM), recovery time.

Scenario C — Phase fault:

- Set `v_in[2] = 0` at all times (phase 3 is dead).
- `omega_ref`: 2400 RPM, `T_load`: 6 N·m.
- Measure: achieved steady-state speed, torque ripple increase.

Scenario	PI — result	SMC — result	Better controller?
A: Rise time (s)	?	?	?
A: Overshoot (%)	?	?	?
A: Settling time (s)	?	?	?
B: Speed dip (RPM)	?	?	?
B: Recovery time (s)	?	?	?
C: Achieved speed (RPM)	?	?	?

Scenario	PI — result	SMC — result	Better controller?
C: Torque ripple pk-pk (N·m)	?	?	?

Typical results: SMC gives 30–50% faster disturbance recovery in Scenario B, and maintains speed within 50 RPM in Scenario C where PI may drop 150–200 RPM. PI generally has less overshoot in Scenario A with careful tuning.

TAS
K 5.3

Reflection questions

Write short answers (3–5 sentences each) in your lab notebook:

12. Why does the SRM produce torque ripple? Why is it worse with 4 phases (Scenario C) than 5 phases?
13. The SMC has a parameter K_{sw} (switching gain). What happens physically in the motor drive if K_{sw} is too small? What happens if it is too large?
14. The anti-windup in the PI controller stops the integrator from growing when at the torque limit. Draw a graph of what the speed response would look like WITHOUT anti-windup during a large speed step. Why is this bad for a tuk-tuk driver?
15. The tuk-tuk has a resolver for angle measurement. The resolver output is quantised to 12 bits (4096 steps per revolution). At 2400 RPM, how many resolver steps occur per millisecond? Does this resolution seem sufficient for smooth commutation?

Lab 6 — The V-Model: Where Does Your Simulation Fit?

No simulation is run in this lab. Instead, you map everything you built in Labs 0–5 onto the V-Model — the engineering process standard used in automotive development (ISO 26262, AUTOSAR). Understanding this structure is what separates a student who can run a script from an engineer who can develop a product.

6.1 What Is the V-Model?

The V-Model is a development process where the left side of the V defines what the system must do (requirements and design), and the right side verifies that it actually does it (testing and integration). Every level on the left has a matching test activity on the right. Nothing reaches the right side without a completed left side.

The shape of the V represents this: you go down the left side getting more detailed, then back up the right side integrating and testing. The bottom of the V is the code and the simulation model — the most detailed artefact.

Level	LEFT SIDE — Definition	RIGHT SIDE — Verification
System	Feasibility study: , GVW 637 kg, 45 km/h, 15% grade, 48 V — requirements derived in Lab 0	Acceptance test: physical tuk-tuk on road, GPS speed logger, power analyser
Sub-system	SRM + inverter + controller specification: rated torque, speed range, resolver accuracy, efficiency	Dynamometer test: motor on test bench, torque sensor, thermal camera, resolver accuracy measurement
Component	Motor design (poles, winding, inductance profile); Inverter design (MOSFET, gate driver, bus cap); Controller design (gains)	Motor inductance measurement (LCR bridge); MOSFET gate-drive test; Controller HIL on AURIX eval board
Module	OpenModelica phase model (Lab 1–2); EmbedSim VectorBlocks — PI, SMC, commutation (Labs 4–5)	Unit test: each VectorBlock with known input → expected output; Modelica model vs analytical step response
Code	MISRA C:2012 generated by EmbedSim CodeGen: embedsim_step.c, embedsim_step.h	Static analysis (MISRA checker); Code review; Branch coverage analysis; Flash to AURIX, compare to Python simulation

6.2 Where Your Lab Work Sits on the V

Every file you created in this course maps to a specific level:

What you built	V-Model level	Left or right side?
Lab 0 power calculation worksheet	System	Left — requirement derivation
SRM_Phase.mo, SRM_Plant_10_8.mo (Labs 1–2)	Module	Left — component model (plant)
FMU export from OpenModelica (Lab 2)	Module / Component boundary	Left — model-based design artefact
Open-loop commutation test (Lab 3)	Module	Right — first verification of plant model

What you built	V-Model level	Left or right side?
SRM_PI_SpeedController (Lab 4)	Module	Left — controller design
PI gain tuning table (Lab 4)	Component	Right — controller parameter verification
SMC implementation (Lab 5)	Module	Left — advanced controller design
Scenario A/B/C results table (Lab 5)	Sub-system	Right — closed-loop performance verification
Reflection questions (Lab 5)	System	Right — reasoning about design trade-offs
embed_sim_srm_controller.c (CodeGen output)	Code	Left — implementation artefact
MISRA checker run on generated C	Code	Right — code-level verification

TASK 6.1

Trace your simulation to a system requirement

Pick any two rows from your Scenario A/B/C results table (Lab 5) and trace each result back up the V to a system requirement from Lab 0. For each result write:

16. The measurement from your simulation (e.g. 'settling time = 1.1 s in Scenario A').
17. The system requirement it verifies (e.g. '0–40 km/h acceleration ≤ 10 s — derived from feasibility study, Lab 0').
18. Whether it passes or fails.
19. If it fails: which block or gain would you change, and why?

A simulation result that cannot be traced to a requirement is engineering entertainment, not engineering. The V-model forces you to ask: why does this number matter?

TASK 6.2

Identify what is missing before hardware

Look at the right side of the V-Model table above. Your lab work covers the Module and sub-system rows. What is NOT yet done before this drive could be installed in a real tuk-tuk? List at least five items:

Missing verification activity	V-Model level	Risk if skipped
1. _____	_____	_____
2. _____	_____	_____
3. _____	_____	_____
4. _____	_____	_____
5. _____	_____	_____

Think about: thermal testing, EMC (electromagnetic compatibility), mechanical vibration, resolver calibration on the actual shaft, fail-safe behaviour when the MCU resets, road test on actual Sri Lankan roads (potholes, monsoon water, dust).

TAS
K 6.3

The EmbedSim CodeGen link

In the real DFC simulation (db42s02_closed_loop_dfc_20k.py — the PMSM example from the EmbedSim project), the CodeGen pipeline works like this:

```
# From db42s02_closed_loop_dfc_20k.py – the real EmbedSim wiring:
ctrl >> cg_start
cg_start >> dfc >> svpwm_pack >> svpwm >> cg_end
cg_end >> motor
# CodeGen boundary: everything between cg_start and cg_end
# is emitted as MISRA C:2012 into embedsim_gen/embedsim_step.c
```

For the SRM tuk-tuk drive, the equivalent boundary would wrap:

```
cg_start >> pi_or_smc >> commutation >> cg_end
# The FMU plant stays OUTSIDE (C_CODEGEN_EXCLUDE = True)
# It is replaced by real hardware on the AURIX
```

Answer these questions in your lab notebook:

20. Why is the FMU plant excluded from CodeGen? What replaces it on the AURIX TC3xx hardware?
21. The generated embedsim_step.c is called every 50 μ s by the AURIX GTM timer interrupt. What would happen if the C code took 60 μ s to execute? How would you detect this on hardware?
22. MISRA C:2012 Rule 14.3 says a controlling expression shall not be invariant. Find one place in your PI or SMC code where this rule would be violated and explain how to fix it.
23. The V-Model requires traceability: every line of code must link to a requirement. What requirement from Lab 0 does the anti-windup logic in the PI controller trace to?

The CodeGen output is not a convenience — it is the reason the simulation has engineering value. A Python simulation that cannot produce verified embedded C code is a demonstration, not a development tool. EmbedSim's CodeGen closes the V: the same algorithm that was verified in simulation is the one that runs on the AURIX.

Quick Reference Card

Key Numbers — Commit These to Memory

Quantity	Value	Why it matters
DC bus voltage	48 V	Touch-safe (<60 V), LiFePO4 pack — same as
Peak phase current	80 A	MOSFET and wire sizing limit
Continuous current	45 A RMS	Thermal limit of windings
Aligned inductance	4.5 mH	Peak flux, maximum torque region
Unaligned inductance	0.9 mH	Phase turn-on region, low back-EMF
Rated speed	2400 RPM	$= 251 \text{ rad/s} \rightarrow v_{\text{vehicle}} = n \times 2\pi \times r_w / (60 \times G) = 38.5 \text{ km/h}$ at rated speed ✓
Rotor pole pitch	45° (0.785 rad)	$360^\circ \div 8 \text{ poles}$
Phase offset	72° (1.257 rad)	$360^\circ \div 5 \text{ phases}$
FMU time step	50 μs	Stable for 20 kHz switching
Vehicle load at 45 km/h	$\approx 12 \text{ N}\cdot\text{m}$	Motor shaft torque: road load at cruise, GVW 637 kg

EmbedSim Cheat Sheet

What you want to do	Code pattern
Load an FMU plant	<code>plant = FmuBlock('motor.fmu', dt=DT)</code>
Connect two blocks	<code>topo.connect(blockA, 'out_name', blockB, 'in_name')</code>
Set a constant input	<code>plant.set_input('T_load', 8.0)</code>
Read an output value	<code>val = plant.get_output('omega_out')</code>
Run the simulation	<code>engine = SimulationEngine(topo, logger, dt, t_end); engine.run()</code>
Plot a signal	<code>logger.plot(['omega_out', 'T_ref'])</code>
Clip a value	<code>x = np.clip(x, lo, hi) # or max(lo, min(x, hi))</code>
Compute sign with smoothing	<code>sat = s/phi if abs(s)<phi else np.sign(s)</code>

OpenModelica Cheat Sheet

Task	How
Declare a parameter	<code>parameter Real Kp = 1.0 "Description [unit]";</code>
Declare a state variable	<code>Real omega(start=0.0) "Speed [rad/s]";</code>
Write a derivative	<code>J * der(omega) = T_net; // means $J \cdot d\omega/dt = T$</code>
Compute modulo	<code>mod(x, y) // like Python $x \% y$</code>

Task	How
Get π	Modelica.Constants.pi
Export FMU	FMI → Export FMU → Co-Simulation 2.0
Validate FMU variables	Unzip .fmu, open modelDescription.xml

Good luck — and remember: simulate first, solder second.

EmbedSim Project • github.com/vectorsim/embed_sim_project